AgriRover-Q — API Communication

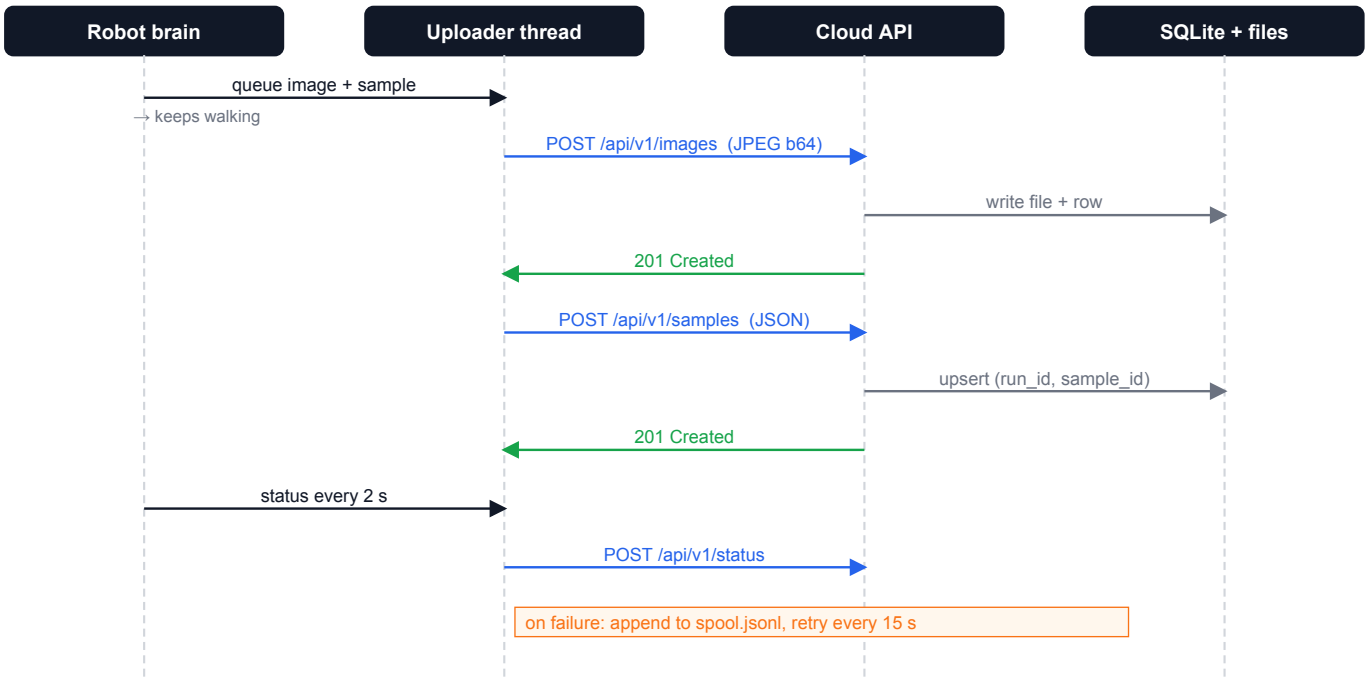
How the robot talks to the cloud: transport, authentication, endpoints, payloads and retry behaviour. API version 0.2; schemas are generated from the FastAPI application (`cloud/api.py`).

1. Overview

Base URL	http://<server>:8000 (simulator: http://127.0.0.1:8000)
Transport	HTTP/1.1, JSON bodies (UTF-8). Use HTTPS behind a reverse proxy in the field.
Authentication	Header `X-API-Key: <key>` on every robot request (server env AGRI_API_KEY, default dev-key). Missing/wrong key → 401.
Robot client	`agrirover/brain/telemetry.py`: Python standard library only, background thread, 3 s timeout, offline spool
Storage	SQLite (samples, images, status) + image files; AGRI_DATA_DIR
Interactive docs	GET /docs (Swagger UI), GET /openapi.json

2. Communication flow

The brain never waits on the network: it queues uploads and keeps walking. A background thread posts them; anything that fails is appended to a JSONL spool and retried every 15 s until accepted. Heartbeats are best-effort and are dropped instead of spooled.



Per sample: crouch → read soil (8 s) → stand → photo → POST /images, then POST /samples. Every 2 s: POST /status.

3. Endpoints

Method	Path	Auth	Success	Purpose
POST	`/api/v1/samples`	key	201	Upload one soil sample (position, soil, air, battery)
POST	`/api/v1/images`	key	201	Upload the sample photo (JPEG, base64)
POST	`/api/v1/status`	key	200	Heartbeat: state, position, battery (every 2 s)
PUT	`/api/v1/fields/{field_id}`	key	200	Set (PUT), read (GET) or delete a field's GPS boundary

Method	Path	Auth	Success	Purpose
GET	`/api/v1/fields/{field_id}`	key	200	Set (PUT), read (GET) or delete a field's GPS boundary
DELETE	`/api/v1/fields/{field_id}`	key	200	Set (PUT), read (GET) or delete a field's GPS boundary
GET	`/api/v1/fields`	key	200	List fields with their GPS boundaries
POST	`/api/v1/fields/{field_id}/contains`	key	200	Geofence check: is a GPS point inside the field?
GET	`/health`	none	200	Liveness check
GET	`/api/v1/runs`	key	200	List survey runs with counts and average moisture
GET	`/api/v1/runs/{run_id}/samples`	key	200	All samples of one run
GET	`/api/v1/runs/{run_id}/status`	key	200	Latest heartbeat of a run
GET	`/api/v1/runs/{run_id}/geojson`	key	200	Samples as GeoJSON points (moisture map)
GET	`/api/v1/images/{filename}`	none	200	Download a sample photo

4. Robot → cloud messages

4.1 POST /api/v1/samples — one soil sample

Sent after each sample. Idempotent on (run_id, sample_id): re-sending a spooled sample overwrites, never duplicates. Response **201** `{"ok": true, "run_id": …, "sample_id": …}`.

Field	Type	Req.	Meaning
`rover_id`	string	yes	Robot identifier from the mission config
`run_id`	string	yes	One survey run (UTC start time on the robot)
`sample_id`	integer	yes	1-based counter within the run; (run_id, sample_id) is unique
`timestamp`	string	yes	ISO-8601 UTC time the sample was recorded
`lane`	integer | null	no	1-based lane (row gap) index on the route
`position`	Position	yes	Fused position: odometry + crop-row camera + GPS
`position.lat`	number	yes	WGS-84 latitude (deg)
`position.lon`	number	yes	WGS-84 longitude (deg)
`position.source`	string | null	no	'fused' for the position estimate
`gps`	Gps | null	no	Raw GPS fix at sample time (null if none)
`gps.lat`	number	yes	WGS-84 latitude (deg)
`gps.lon`	number	yes	WGS-84 longitude (deg)
`gps.fix`	string	yes	'none' | '2D' | '3D' | 'RTK'
`gps.hdop`	number	yes	Horizontal dilution of precision
`gps.satellites`	integer	yes	Satellites used
`heading_deg`	number | null	no	Compass heading, 0 = north, clockwise
`soil`	Soil	yes	Soil probe readings
`soil.moisture_vwc`	number | null	no	Volumetric water content (m³/m³); null if the probe failed
`soil.moisture_raw`	integer	yes	ADS1115 counts (gain 1) before calibration
`soil.temperature_c`	number | null	no	Degrees Celsius
`soil.probe_ok`	boolean	no	False if the probe read like air (did not enter the soil)
`soil.probe_depth_mm`	number | null	no	Probe depth from the crouch kinematics
`air`	Air	yes	BME280 readings
`air.temperature_c`	number	yes	Degrees Celsius
`air.humidity_pct`	number	yes	Relative humidity (%)
`air.pressure_hpa`	number	yes	Barometric pressure (hPa)

Field	Type	Req.	Meaning
`battery`	Battery	yes	Pack voltage and estimated state of charge
`battery.voltage`	number | null	no	Battery pack voltage (V)
`battery.soc_pct`	number | null	no	State of charge estimated from loaded voltage (%)
`image`	string | null	no	Filename of the photo uploaded to /api/v1/images

Example: real payload from completed simulator run video-20261110-100504


```
{
  "rover_id": "agrirover-q-01",
  "run_id": "video-20261110-100504",
  "sample_id": 1,
  "timestamp": "2026-11-10T04:05:35+00:00",
  "lane": 1,
  "position": {
    "lat": 41.9853,
    "lon": -93.6975816,
    "source": "fused"
  },
  "gps": {
    "lat": 41.9853239,
    "lon": -93.697571,
    "fix": "3D",
    "hdop": 0.98,
    "satellites": 12
  },
  "heading_deg": 92.7,
  "soil": {
    "moisture_vwc": 0.2838,
    "moisture_raw": 12126,
    "temperature_c": 18.31,
    "probe_ok": true,
    "probe_depth_mm": 50.0
  },
  "air": {
    "temperature_c": 24.08,
    "humidity_pct": 56.6,
    "pressure_hpa": 1009.5
  },
  "battery": {
    "voltage": 8.18,
    "soc_pct": 93.0
  },
  "image": "video-20261110-100504_s001.jpg"
}
```


4.2 POST /api/v1/images — sample photo

JPEG (640×360 from the head camera, taken standing, crop rows in view), base64 in JSON. Only .jpg/.jpeg/.png accepted; any path in the filename is stripped. Response **201** `{"ok": true, "url": "/api/v1/images/<file>"}`.

Field	Type	Req.	Meaning
`run_id`	string	yes	One survey run (UTC start time on the robot)
`sample_id`	integer	yes	1-based counter within the run; (run_id, sample_id) is unique
`filename`	string	yes	JPEG filename (no path)
`content_type`	string	no	'image/jpeg'
`data_b64`	string	yes	Base64-encoded image bytes

4.3 POST /api/v1/status — heartbeat

Every 2 s while running, plus a final one when the mission ends. The server keeps the latest per run. Response **200**.

Field	Type	Req.	Meaning
`rover_id`	string	yes	Robot identifier from the mission config

Field	Type	Req.	Meaning
`run_id`	string	yes	One survey run (UTC start time on the robot)
`timestamp`	string	yes	ISO-8601 UTC time the sample was recorded
`state`	string	yes	Mission state (see section 7)
`lane`	integer | null	no	1-based lane (row gap) index on the route
`samples`	integer	no	Samples recorded so far in the run
`position`	Position | null	no	Fused position: odometry + crop-row camera + GPS
`position.lat`	number	yes	WGS-84 latitude (deg)
`position.lon`	number	yes	WGS-84 longitude (deg)
`position.source`	string | null	no	'fused' for the position estimate
`heading_deg`	number | null	no	Compass heading, 0 = north, clockwise
`battery`	Battery | null	no	Pack voltage and estimated state of charge
`battery.voltage`	number | null	no	Battery pack voltage (V)
`battery.soc_pct`	number | null	no	State of charge estimated from loaded voltage (%)

Example: final heartbeat of the same run

```
{
  "rover_id": "agrirover-q-01",
  "run_id": "video-20261110-100504",
  "timestamp": "2026-11-10T04:11:05+00:00",
  "state": "done",
  "lane": 3,
  "samples": 9,
  "position": {
    "lat": 41.985311,
    "lon": -93.6974754,
    "source": null
  },
  "heading_deg": 93.4,
  "battery": {
    "voltage": 8.06,
    "soc_pct": 88.0
  }
}
```

5. Field boundary (GPS geofence)

The operator (or a farm-management app) registers each field's GPS boundary before a survey. The boundary is a polygon of [lat, lon] corner points listed in order around the field. The server validates it and returns its area, perimeter, centroid and a GeoJSON polygon ready for mapping tools.

Method	Path	Purpose
PUT	`/api/v1/fields/{field_id}`	Create or replace the boundary (200)
GET	`/api/v1/fields/{field_id}`	Read one field (404 if unknown)
GET	`/api/v1/fields`	List all fields
DELETE	`/api/v1/fields/{field_id}`	Remove a field
POST	`/api/v1/fields/{field_id}/contains`	Geofence check for one point: {lat, lon} → inside + distance to edge

Request body (PUT)

Field	Type	Req.	Meaning
`name`	string	yes	Human-readable field name
`boundary`	array	yes	[lat, lon] corner points in order around the field
`crop`	string | null	no	Crop planted, e.g. 'maize'
`row_spacing_m`	number | null	no	Distance between crop rows (m)

Field	Type	Req.	Meaning
`row_heading_deg`	number | null	no	Compass direction the rows run (deg)
`notes`	string | null	no	Free text

Validation (rejected with 422): fewer than 3 distinct points; latitude outside ±90 or longitude outside ±180 (this catches swapped [lon, lat]); edges that cross (points not listed in order around the field); area under 1 m² or over 10 km². The closing point may be repeated or omitted.

Example: set the demo field boundary

```
PUT /api/v1/fields/demo-plot
X-API-Key: dev-key

{
  "name": "Demo plot, Ames IA",
  "boundary": [
    [41.9852892, -93.6976121],
    [41.9852892, -93.6974671],
    [41.9853216, -93.6974671],
    [41.9853216, -93.6976121]
  ],
  "crop": "maize",
  "row_spacing_m": 0.6,
  "row_heading_deg": 90.0
}
```

Response 200 (shortened)

```
{
  "field_id": "demo-plot",
  "name": "Demo plot, Ames IA",
  "crop": "maize",
  "row_spacing_m": 0.6,
  "row_heading_deg": 90.0,
  "area_m2": 43.3,
  "perimeter_m": 31.2,
  "centroid": {
    "lat": 41.9853054,
    "lon": -93.6975396
  },
  "updated": "2026-11-10T04:33:27+00:00",
  "boundary": "[... 4 points ...]",
  "geojson": {
    "type": "Feature",
    "geometry": {
      "type": "Polygon",
      "coordinates": "[[[lon, lat], ...]]"
    }
  }
}
```

Example: geofence check

```
POST /api/v1/fields/demo-plot/contains   {"lat": 41.9853054, "lon": -93.6975396}
-> {"field_id": "demo-plot", "inside": true, "distance_to_edge_m": 1.8}

POST /api/v1/fields/demo-plot/contains   {"lat": 41.9853054, "lon": -93.6974308}
-> {"field_id": "demo-plot", "inside": false, "distance_to_edge_m": 3.0}
```

Robot side: implemented vs planned

Item	Status
Boundary storage, validation, area / perimeter / GeoJSON, geofence check	Implemented (cloud/api.py), covered by tests
Mission route lanes must lie inside the field boundary	Planned: check when the mission JSON is generated
Robot downloads the boundary at start (GET /api/v1/fields/{id})	Planned: not yet in the brain
Robot stops and sits if its fused position leaves the boundary	Planned: not yet in the brain

6. Reading data back

Request	Returns
GET /api/v1/runs	All runs: sample count, first/last timestamp, average VWC, last state
GET /api/v1/runs/{run_id}/samples	Every sample of a run (same JSON the robot sent)
GET /api/v1/runs/{run_id}/status	Latest heartbeat
GET /api/v1/runs/{run_id}/geojson	FeatureCollection of sample points (vwc, soil temp, image) for QGIS / geojson.io
GET /api/v1/images/{filename}	The JPEG (no key needed, so it opens in a browser)
GET /health	{"ok": true}: liveness check, no key

Run video-20261110-100504 in the bundled database: 9 samples, mean VWC 0.281.

curl examples

```
# list runs
curl -H 'X-API-Key: dev-key' http://127.0.0.1:8000/api/v1/runs

# moisture map of one run
curl -H 'X-API-Key: dev-key' http://127.0.0.1:8000/api/v1/runs/<run_id>/geojson > map.geojson

# post a sample by hand (what the robot does)
curl -X POST http://127.0.0.1:8000/api/v1/samples \
     -H 'Content-Type: application/json' -H 'X-API-Key: dev-key' \
     -d '{"rover_id":"r1","run_id":"test","sample_id":1,"timestamp":"2026-11-10T09:00:00Z",
          "position":{"lat":41.98531,"lon":-93.69758},"soil":{"moisture_vwc":0.27,"moisture_raw":14320},
          "air":{"temperature_c":22.1,"humidity_pct":61,"pressure_hpa":1009.8},"battery":{}}'
```

7. Status values (mission states)

state	Meaning
boot	Standing up
enter	Walking to the start of a lane
follow	Walking the lane, steering on the crop rows (camera)
stop	Stopping at a sample point
crouch	Lowering the body: belly probe goes into the soil
sample	Reading soil moisture + temperature (8 s dwell)
rise	Standing, taking the photo, queueing uploads
exit	Walking out past the row ends
headland	Crossing to the next lane
align	Turning to face down the next lane
done	Route finished, sitting
abort	Stopped early (low battery), sitting

8. Errors and robot behaviour

HTTP	When	What the robot does
401	Missing or wrong X-API-Key	Logs it; samples and images go to the spool and retry every 15 s
400	Image not .jpg/.jpeg/.png	Logged (should not happen; the robot always sends JPEG)
422	Field boundary invalid (< 3 points, edges cross, lat/lon swapped, area out of range)	Operator request: fix the boundary and resend
404	Unknown run / image on GET	(read side only)
422	Body fails schema validation	Logged and spooled; fix the client, the spool resends
—	Timeout (3 s) / no network	Spooled to data/spool/<run_id>/pending.jsonl; photos also kept locally
5xx	Server error	Spooled and retried